Experiment No -06 (Group B)

Title:-

Real-Time Event Processing System Using Queue Data Structure

Problem statement:-

A call center receives incoming calls, and each call is assigned a unique customer ID. The calls are answered in the order they are received. Your task is to simulate the call queue of a call center using a queue data structure.

- addCall(customerID, callTime): Add a call to the queue with the customer ID and the call time (in minutes).
- answerCall(): Answer and remove the first call from the queue.
- viewQueue(): View all calls currently in the queue without removing them.
- isQueueEmpty(): Check if the queue is empty.

Pre-requisite:-

- Basic knowledge of **Python programming.**
- Understanding of **functions** in Python.
- The Queue data structure and its foundational FIFO operations (enqueue and dequeue).

Objective:-

The primary objective is to simulate a **sequential call center operations model** using a **Queue data structure**.

The system ensures customer service equity by answering calls in the exact order they arrive (**First-In, First-Out**).

It targets maximum operational efficiency by enabling agents to log calls, answer waiting customers, and inspect queue volume instantaneously without altering the pipeline order.

Software and Hardware requirements:-

- **Operating System recommended:-** 64-bit Open source Linux or its derivative
- **Programming tools recommended: -** Python 3
- **Programming IDE like:** Any (IDLE, Jupyter, VS Code, etc.)
- **Hardware:** Standard PC with keyboard input

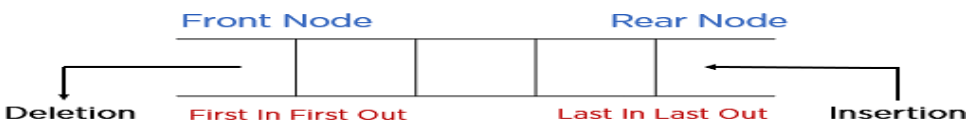
Outcome:-

Running the simulation generates a predictable, organized log that mirrors real-life support operations:

- **Ordered Entry**: Incoming calls from CUST_001, CUST_002, and CUST_003 are successfully sequenced based on arrival timestamps.
- **Fair Dispatching**: The answerCall() operation strictly connects agents to CUST_001 first, avoiding any "line-skipping" issues.
- **Real-time Metrics**: Managers can view exactly how many callers are left in line (and their estimated talk times) without disrupting the queue data.
- **Idle Validation**: When all calls are answered, isQueueEmpty() returns True, signalling that agents are free to take a break or handle other tasks.

Theory:-

- A queue is like a "natural" queue of elements. It is an ordered list in which all insertions occur at one end called the back or rear of the queue, whereas all deletions occur at the other end called the front or head of the queue.
- In the popular terminology, insertion and deletion in a queue are respectively called the enqueue and the dequeue operations.
- The element dequeued from a queue is always the first to have been enqueued among the elements currently present in the queue.
- In view of this, a queue is often called a First-In-First-Out or a FIFO list.



Queue Representation

Basic concept of Queue

1. A queue can be defined as an ordered list which enables insert operations to be performed at one end called REAR and delete operations to be performed at another end called FRONT.
2. Queue is referred to be as First In First Out list.
3. For example, people waiting in line for a rail ticket form a queue.



Algorithm / Steps:

1. **Initialize** an empty queue using collections.deque.
2. Provide a **menu** with the following options:
    - Add a new event
    - Process the next event
    - Display all pending events
    - Cancel an event
    - Exit the program
3. **Add Event:** Append the event to the end of the queue.
4. **Process Next Event:** Remove and display the event at the front of the queue.
5. **Display Events:** Print all events from the front to rear of the queue.
6. **Cancel Event:** Remove a specific event by name from the queue.
7. **Exit:** Stop the program.

Conclusion:-

This experiment successfully demonstrates the use of the queue data structure for managing real-time events. By implementing core queue operations (enqueue, dequeue, and remove), the system efficiently handles dynamic user-driven inputs, simulating a real-world event scheduler.

Viva Questions:

1. What is a queue?
2. What is the difference between stack and queue?
3. What are real-world examples of queues?
4. Why is `deque` used instead of a list in Python?
5. What is the time complexity of `enqueue` and `dequeue` operations in a deque?